

 PYTHON 3.x UTILITY

ALL IN ONE COLLEGE TOOL

FOR VIT-B STUDENTS

Automating 75% Attendance Safety, Relative Grades, Target SGPA & Hostel Spending



75% Attendance
Safe Bunk Tracker



Target SGPA
Semester Planner



Relative Grading
Table-5 & FAT Rule



Hostel Budget
Pocket Money Log



ALL IN ONE COLLEGE TOOL FOR VIT – B STUDENTS

An all-in-one terminal to track attendance ,
crack relative grading,
and keep hostel expenses in check.

- Course : Introduction to problem solving and programming (CSE1021)
- By : KRISHNA GAUTAM
- REG.NO : 26BCE10251
- PROF. : DR.MURUGESHWARI .K



VIT- BHOPAL

PROBLEM STATEMENT

- **The 75% Rule :** We all want to extend trips back home or skip early 8:30 AM lectures, but nobody can afford a debarment notice before FATs. Manual calculation of how many extra days we can safely stay home leads to risky guesswork.
- **Chasing CGPA Targets:** We know what CGPA we need for placements or branch goals, but calculating the exact SGPA needed next semester requires sitting down with pen and paper every time.
- **Decoding Relative Grading:** Theory course marks are scattered across CAT-1, CAT-2, 3 DAs, and FAT. Scaling internals out of 60, remembering the mandatory 40% raw FAT cutoff, and figuring out what grade we will get using class Mean (μ) and standard deviation (σ) gives every student a headache.
- **Hostel Pocket Money Disappearing:** Between quick canteen snacks, printing lab reports, and stationery, our monthly budget drains before the month even ends. Without an instant daily tracker, we end up broke with zero idea where the cash went.

The Solution:

Instead of relying on guesswork or messy notebook calculations, we built the **ALL IN ONE COLLEGE TOOL**—a fast, lightweight terminal app that handles attendance safety, grade predictions, SGPA planning, and hostel budget logs in a single click.

OBJECTIVES:

- **One Single Terminal Utility:** Replace messy notebook calculations and random formulas with a unified, lightweight Python tool.
- **Zero Dependencies Needed:** Built to run straight out of the box on standard Python without downloading extra packages.
- **Academic Safety Nets:** Enforce institutional hard rules right in the code (mandatory 40% FAT cutoff and the 75% attendance threshold).
- **Instant Answers, Zero Guesswork:** Give students immediate answers on whether they can safely bunk, what grade they will get, and what SGPA to target.

Functional requirements

- **Attendance & Bunk Safety (Module 1):** Takes conducted and attended lectures, verifies that attended < total, calculates percentage, and determines integer safe bunks (>75%) or required recovery lectures (< 75%) using integer arithmetic.
- **Semester Target SGPA Planning (Module 2):** Takes current CGPA, completed semesters, and target CGPA; computes required next-sem SGPA and alerts if it exceeds the 10.0 .
- **Theory Scaling & Grade Prediction (Module 3):** Normalizes CAT-I (15%), CAT-II (15%), and 3 DAs (30%) out of 60; enforces the non-negotiable 40% raw FAT pass rule; assigns letter grades using Relative cutoffs (μ and σ).
- **Hostel Pocket Money Manager (Module 4):** Sets monthly budget, appends itemized expense entries with descriptions, calculates real-time remaining balance, and triggers overdraft warnings.

Non – Functional requirements :

- **Beginner-Friendly Usability:** Clean, numbered console interface (1 to 5) with plain English prompts, formatted divider bars, and clear instructions requiring zero technical manual.
- **Crash-Proof Reliability:** Built with defensive validation (`.strip()`, `.isdigit()`, and `try-except ValueError`) so invalid inputs or empty presses never crash the terminal during an active session
- **Instant Performance:** Operates entirely in memory with basic arithmetic and list operations, delivering instantaneous calculations without latency or server wait times.
- **Universal Portability:** Pure Python 3.x implementation with zero third-party dependencies (pip), running out of the box on Windows, macOS, or Linux systems.

SYSTEM ARCHITECTURE:

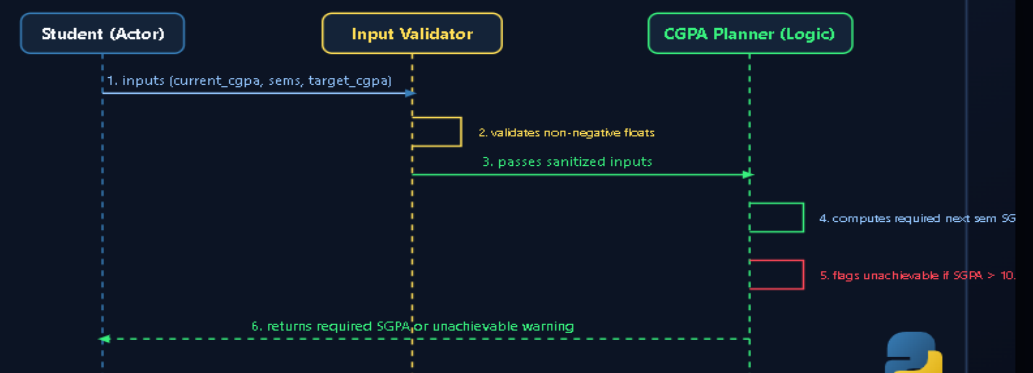
- **Data Storage** : Dynamic memory registers (`monthly_budget`, `total_spent`) and synchronized lists (`expense_names`, `expense_amounts`) for real-time tracking.
- **Presentation** : A clean, persistent console menu running on an infinite `while` loop inside `main()`.
- **Defensive Gateway**: Centralized input parsing verifying positive values and boundary limits before reaching core math routines.
- **application Processing** : Four independent functional modules (`attendance()`, `cgpa_target()`, `evaluator()`, `budget()`) encapsulating institutional formulas and boundary checks.

DESIGN DIAGRAMS :

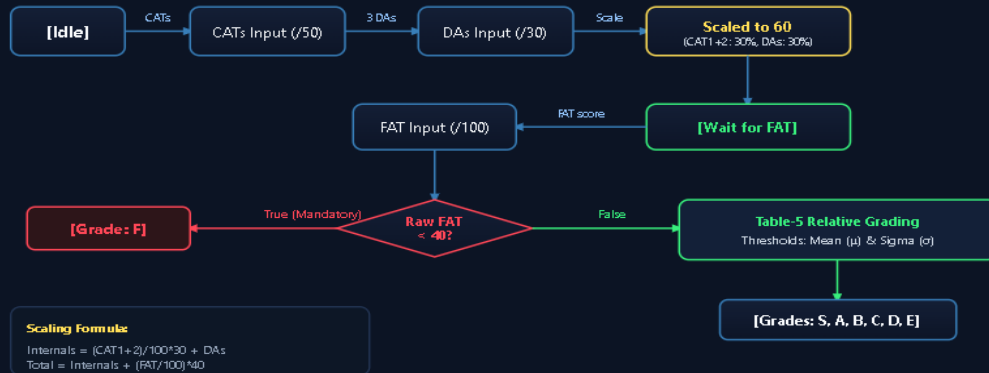
MODULE 1: ATTENDANCE ELIGIBILITY WORKFLOW



MODULE 2: TARGET SGPA PLANNER SEQUENCE



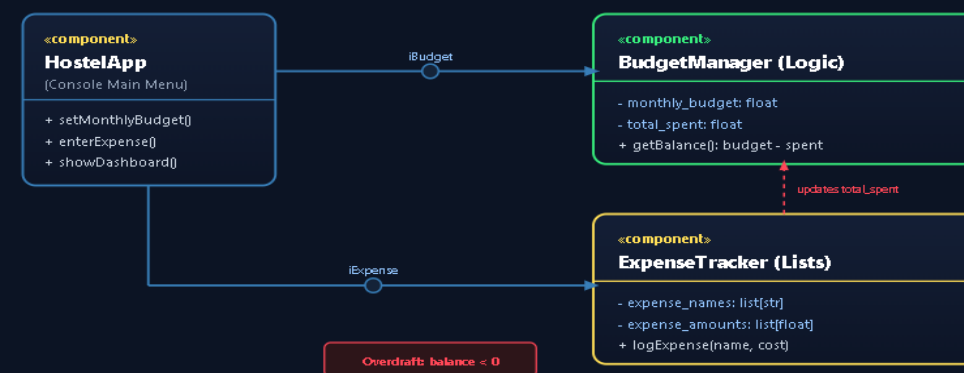
MODULE 3: EXAM EVALUATION STATE DIAGRAM



Scaling Formula:

Internals = (CAT1+2)/100*30 + DAs
Total = Internals + (FAT/100)*40

MODULE 4: HOSTEL BUDGET COMPONENT DIAGRAM



Overdraft: balance < 0

DESIGN DECISION

- **Integer Arithmetic for Safe Bunks:** Uses floor division so students never dip below the 75% boundary.
- **VIT Cutoffs:** Implements dynamic mean (μ) and standard deviation (σ) modeling rather than fixed absolute thresholds, aligning with real institutional grading.
- **Mandatory FAT Check:** Explicitly checks if raw FAT < 40, immediately assigning an **F Grade** regardless of a high internal score, preventing false grade predictions.
- **In-Memory Decision:** Prioritizes privacy, speed, and cross-platform compatibility without file locks or external database drivers.

Implementation Details

- **Modular Procedural Design:** Structured into user-defined functions with parameter passing and clean return points.
- **Conditional Branching:** Multi-tier `if-elif-else` constructs to accurately capture Gaussian relative cutoffs and passing rules.
- **Dynamic Data Structures:** Python lists used for parallel tracking of descriptions and numerical costs with real-time accumulation.
- **Exception & Input Sanitization:** Defensive programming through `try-except ValueError` blocks, `.isdigit()`, and `.strip()` string sanitization.

Screenshots / Results

```
C:\windows\py.exe X + v

=====
ALL IN ONE COLLEGE TOOL FOR VIT- B STUDENTS
=====
1. HOW MANY CLASSES STUDENT CAN BUNK SATYING ABOVE 75%/CLASSES TO ATTEND TO STAY ABOVE 75 %
2. Next Semester SGPA Target SETTER
3. EXAM EVALUATION, TEE GOAL & GRADE PREDICTOR
4. Pocket Money & Daily Expense Log
5. Exit Application

Enter choice (1-5): 1

--- HOW MANY CLASSES STUDENT CAN BUNK SATYING ABOVE 75%/CLASSES TO ATTEND TO STAY ABOVE 75 % ---
Enter total lectures conducted till today: 33
Enter lectures you attended: 31
Your Current Attendance: 93.94%
Status: ELIGIBLE (Safe to sit/appear for TEE)
You can safely bunk 8 more lecture(s) while staying >= 75%.

=====
ALL IN ONE COLLEGE TOOL FOR VIT- B STUDENTS
=====
1. HOW MANY CLASSES STUDENT CAN BUNK SATYING ABOVE 75%/CLASSES TO ATTEND TO STAY ABOVE 75 %
2. Next Semester SGPA Target SETTER
3. EXAM EVALUATION, TEE GOAL & GRADE PREDICTOR
4. Pocket Money & Daily Expense Log
5. Exit Application

Enter choice (1-5): 2

--- Target SGPA & CGPA SETTER FOR NEXT SEM ---
Enter current CGPA (e.g., 7.5): 8
Enter completed semesters (e.g., 1): 2
Enter target overall CGPA: 8.5
Target Overall CGPA: 8.5
Result: You must secure at least 9.5 SGPA in the next semester.

=====
ALL IN ONE COLLEGE TOOL FOR VIT- B STUDENTS
=====
1. HOW MANY CLASSES STUDENT CAN BUNK SATYING ABOVE 75%/CLASSES TO ATTEND TO STAY ABOVE 75 %
```

```
C:\windows\py.exe X + v

=====
ALL IN ONE COLLEGE TOOL FOR VIT- B STUDENTS
=====
1. HOW MANY CLASSES STUDENT CAN BUNK SATYING ABOVE 75%/CLASSES TO ATTEND TO STAY ABOVE 75 %
2. Next Semester SGPA Target SETTER
3. EXAM EVALUATION, TEE GOAL & GRADE PREDICTOR
4. Pocket Money & Daily Expense Log
5. Exit Application

Enter choice (1-5): 3

--- VIT Bhopal Exam Weightage & Relative Grade Predictor ---
CAT-I: 50 Marks (Scaled to 15%)
CAT-II: 50 Marks (Scaled to 15%)
Digital Assignments (3 x 10): 30 Marks
Final Assessment Test (FAT): 100 Marks (Scaled to 40%)

-----
Enter CAT-I marks (out of 50): 38
Enter CAT-II marks (out of 50): 40
Enter Digital Assignment 1 marks (out of 10): 10
Enter Digital Assignment 2 marks (out of 10): 10
Enter Digital Assignment 3 marks (out of 10): 8

>> Internal Marks Secured: 51.4 / 60
Minimum FAT Score Needed Just to Pass: 40.0 / 100

--- Predict Your Final Course Grade (Table-5 Relative Grading) ---
Enter your expected or scored FAT marks (out of 100): 80

----- SCORE BREAKDOWN -----
Internal Contribution (CATs + DA): 51.4 / 60
FAT Contribution: 32.0 / 40
Grand Total Marks: 83.4 / 100

[Optional: Class Statistics for Relative Grading]
Enter class Mean (press Enter for default 60):
Enter class Sigma / Std Dev (press Enter for default 10):

----- FINAL GRADE ASSIGNMENT -----
Assigned Grade: A (Excellent)
Grade Points: 9
```

```
C:\windows\py.exe X + v

Enter choice (1-5): 4

--- Hostel Budget & Expense Log ---
Monthly Pocket money: Rs. 0
Total Expended: Rs. 0
Remaining Balance: Rs. 0

-----
1. Set / Update Monthly Pocket money
2. Add Expense (Canteen, Printing, other)
3. View Detailed Expense Log
4. Return to Main Menu
Enter choice (1-4): 1
Enter your monthly pocket money amount (Rs.): 500
monthly budget updated successfully.

--- Hostel Budget & Expense Log ---
Monthly Pocket money: Rs. 500
Total Expended: Rs. 0
Remaining Balance: Rs. 500

-----
1. Set / Update Monthly Pocket money
2. Add Expense (Canteen, Printing, other)
3. View Detailed Expense Log
4. Return to Main Menu
Enter choice (1-4): 2
Expense description: chips
Amount spent (Rs.): 50
Recorded: chips (-Rs. 50)

--- Hostel Budget & Expense Log ---
Monthly Pocket money: Rs. 500
Total Expended: Rs. 50
Remaining Balance: Rs. 450

-----
1. Set / Update Monthly Pocket money
2. Add Expense (Canteen, Printing, other)
3. View Detailed Expense Log
4. Return to Main Menu
Enter choice (1-4): 3

--- Expense History ---
```

Testing Approach

- **Boundary Value Testing:** Verified boundary conditions for attendance (attended classes equal to total conducted, attended classes set to 0).
- **Institutional Rule Verification:** Tested edge cases like scoring 60/60 in internal assessments but 38/100 in the raw FAT to ensure the mandatory fail rule triggers an immediate **F Grade**.
- **Mathematical Limit Checks:** Tested SGPA planner with unrealistic targets (e.g., aiming for 9.8 CGPA from 6.0 in one semester) to verify that target limits > 10.0 are flagged as impossible.
- **Negative Balance & Overdraft Checks:** Verified budget operations when spending exceeds available balance to confirm overdraft warnings appear immediately

Challenges Faced

- **Translating Institutional Gaussian Formulas:** Mapping grading boundaries into strictly non-overlapping programmatic conditions without logic gaps.
- **Handling Fractional Bunk Dilemmas:** Constructing exact integer floor division math to prevent rounding up, which could accidentally suggest an unsafe bunk.
- **Sanitizing Multi-Type Terminal Inputs:** Safely handling unexpected student inputs (letters, negative integers, blank enter keys) across both float and integer prompts without breaking program execution

Learnings & Key Takeaways

- **Turning Academic Guidelines into Code:** Learned how to convert real-world university policies—like relative grading distributions and the 75% attendance threshold—into exact mathematical algorithms without logic gaps.
- **Defensive Coding in Practice:** Gained hands-on experience handling real user errors using input sanitization (`.strip()`), numeric checks (`.isdigit()`), and `try-except` blocks to ensure the terminal session never crashes unexpectedly.
- **Modular Code Architecture:** Mastered separating independent functional routines (`attendance`, `cgpa_target`, `evaluator`, `budget`) while linking them smoothly through a centralized `main()` menu loop.
- **Professional Engineering Workflow:** Learned end-to-end software delivery—from writing and verifying functional scripts to structuring technical documentation (`README.md`, `statement.md`) and maintaining version control on GitHub.

Future Enhancements

- **Persistent Data Storage:** Integrate JSON or SQLite lightweight file storage so logged expenses and attendance histories remain saved even after closing the terminal.
- **Interactive Graphical UI:** Build a modern desktop interface using Tkinter or PyQt, or a responsive web dashboard for students on mobile devices.
- **Timetable & VTOP Sync:** Add automated PDF or portal parsing to extract registered course codes, class counts, and attendance directly without manual typing.
- **Predictive Semester Insights:** Implement Multi-Course SGPA and CGPA simulators to model different grade scenarios across an entire semester's credit load.